

Act01 - Búsqueda con GUI



Análisis de Algoritmos

Axel Gutierrez Vazquez 220575328

Profesor:

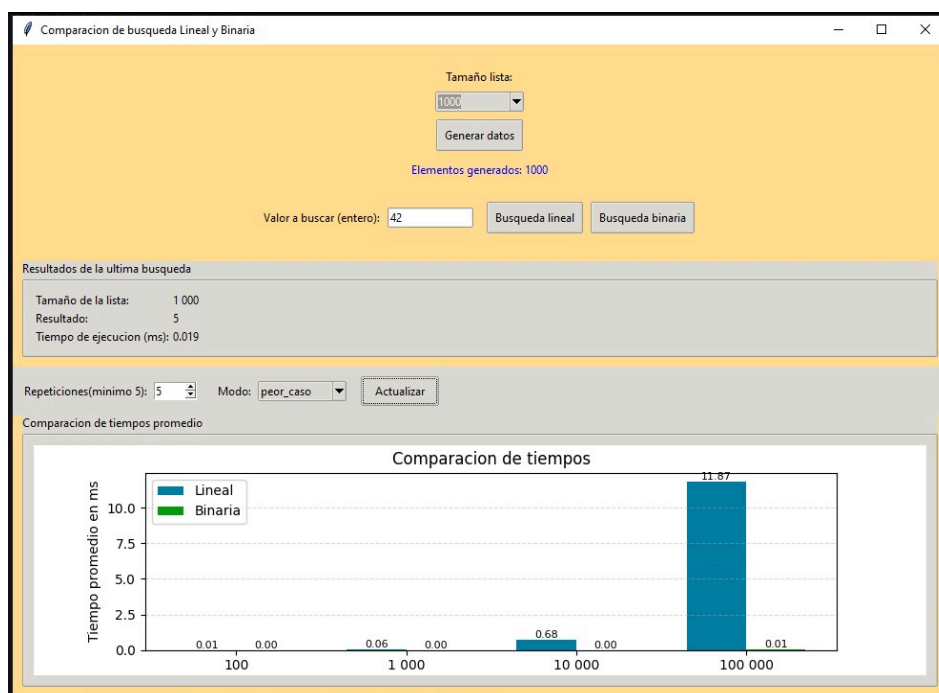
JORGE ERNESTO LOPEZ ARCE DELGADO

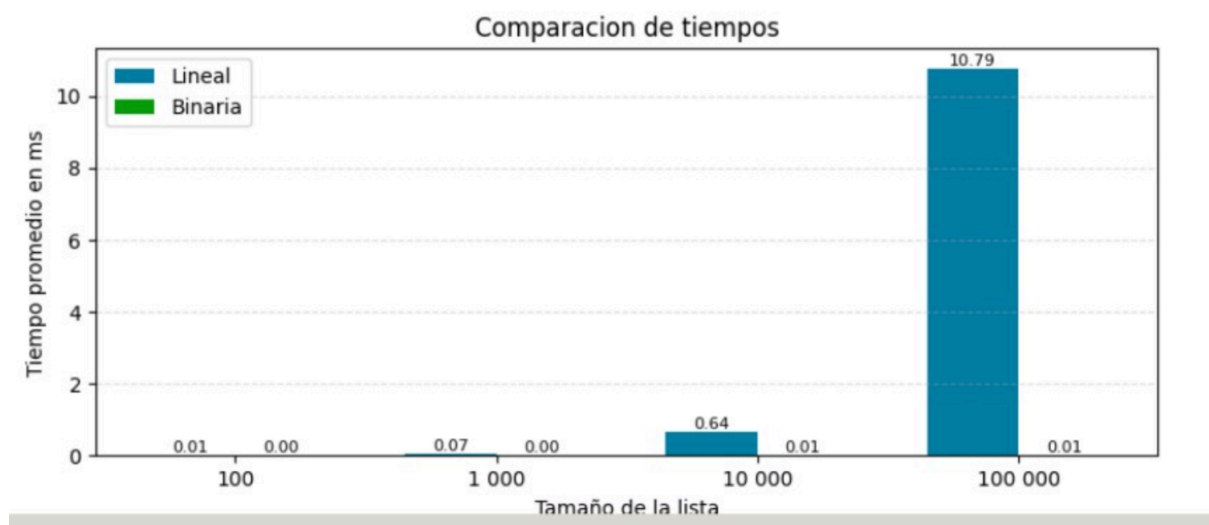
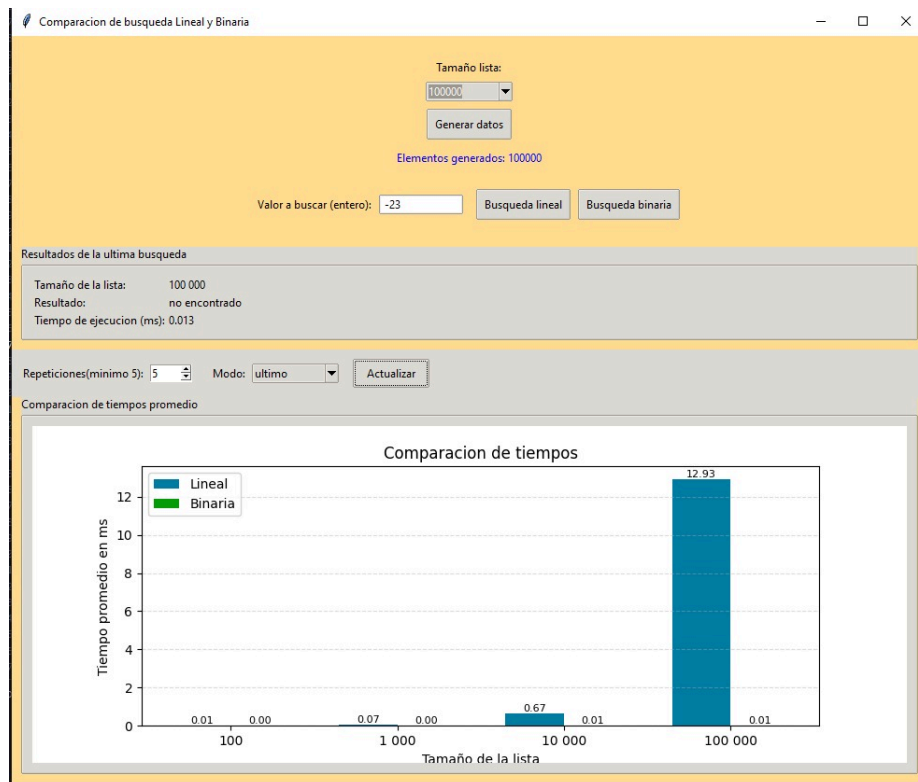
Reporte:

Se desarrolló un código para poner a prueba los tiempos de ejecución de recorridos lineales y binarios; para esto se crearon listas cuyo tamaño variaba entre 100, 1 000, 10 000 y 100 000. La idea era buscar un número dentro de cada lista usando ambos métodos y comparar los tiempos promedio. Para garantizar reproducibilidad se fijó la semilla (`random.seed(42)`) y, en cada repetición, generó una nueva lista ordenada mediante `random.sample(range(size * 10), size)` y `sample.sort()` para simular variabilidad en los datos reales. Se usaron dos modos de búsqueda: `peor_caso` (busco un valor fuera del rango: $\text{size} * 10 + 1$) y `último` (busco `data[-1]`, el último elemento), que permiten ver claramente el comportamiento cuando la lineal debe recorrer casi toda la lista frente a la binaria que reduce el espacio de búsqueda rápidamente.

Para la medición se utilizó `time.perf_counter()` alrededor de cada llamada al algoritmo, convirtiendo la diferencia a milisegundos. Para cada tamaño se ejecutaron varias repeticiones y se calculó el promedio de los tiempos por tamaño y por algoritmo. En la práctica observé que para listas pequeñas (100 y 1 000) ambos algoritmos registran tiempos muy bajos, pero a medida que aumentamos n la diferencia se amplifica: la búsqueda lineal crece aproximadamente de forma proporcional a n , mientras que la binaria se mantiene en valores muy bajos porque el número de comparaciones escala.

En la visualización la barra de la búsqueda lineal domina la figura para tamaños grandes y la binaria casi no se percibe cuando el eje Y está en escala lineal. Esto no significa que la binaria no haga trabajo: simplemente sus tiempos son órdenes de magnitud menores, por lo que, quedan aplastados en una gráfica lineal.





Conclusiones:

Desarrollando esta actividad pude comprender y observar de forma práctica los comportamientos de los métodos de búsqueda lineal y binaria. Al implementar ambos algoritmos, generar listas de distintos tamaños y medir tiempos reales, confirmé que la búsqueda binaria es mucho más eficiente que la lineal para listas ordenadas: su tiempo de ejecución crece muy lentamente y, en la práctica, se mantiene en valores muy pequeños incluso cuando la lista alcanza decenas de miles de elementos. Esa eficiencia se traduce en un ahorro de tiempo evidente cuando se realizan búsquedas repetidas sobre colecciones grandes.